

Codex 上下文工程与 Skills/MCP 位置

SSS & Codex

2026 年 6 月 29 日

Codex 初步 系统提示词注入 Context 管理 Skills / MCP 暴露

视频作者/频道：五道口纳什

发布日期：2026 年 3 月 8 日

视频时长：8:04

视频链接：<https://www.bilibili.com/video/BV1fyNKz5Egb/>

目录

1 从 Context Engineering 看 Codex	2
1.1 为什么观察 TUI session 的 JSONL	3
1.2 本章小结	4
2 合法 prompt injection: agent runtime 怎样注入上下文	4
2.1 session level 注入 Skills 与 context files	5
2.2 message role: 每条消息的位置标签	6
2.3 本章小结	7
3 Skills 与 MCP 在消息结构里的位置	7
3.1 Skills: 渐进式加载的操作说明	7
3.2 MCP: 通过 tools 暴露的外部能力	8
3.3 二者怎样共同塑造 agent 行为	8
3.4 本章小结	9
4 一次 Codex session 的 message history 拆解	9
4.1 System、Developer、User 的分层	9
4.2 turn、assistant 与 compact	10
4.3 AGENTS.md 与 environment context	10
4.4 system prompt sections 与协作模式	11
4.5 本章小结	12
5 总结与延伸	12
5.1 给实践者的三个 takeaways	13
5.2 延伸理解: 为什么 Skills 与 MCP 值得分开设计	13
5.3 最终压缩	13

1 从 Context Engineering 看 Codex

这一章覆盖视频的开场段落, 时间范围是 00:00:00–00:01:14。讲者先把问题框定为: 怎样从 Context Engineering 的角度理解 Codex 这种 modern coding agent。这个角度很关键, 因为 Codex 的能力不能只归因于底层模型; 运行时怎样组织 system 指令、developer 规则、用户请求、项目文件、工具结果和历史摘要, 同样决定了 agent 能走多远、能否守住边界、能否连续完成软件开发任务。

本集的问题意识

Context Engineering 可以理解为“给模型准备工作台”的工程过程: 模型本身像推理引擎, Codex runtime 则负责把任务说明、权限边界、工具回传、项目规则和历史状态摆到正确位置。摆放顺序、可信来源和压缩策略都会改变模型下一步看到的世界。

讲者在 00:00:05–00:00:15 说明, 本集会通过 Context Engineering 分析 Codex 这个现代 web/coding 工具。这里的 Context Engineering 指一套围绕模型输入的组织方法。它关心的问题包括: 哪些内容进入 message history, 哪些内容保持在本地工具层, 哪些内容需要在下一轮继续保留, 哪些内容可以在 compact 之后只留下摘要。

可以把一次 agent loop 想成一次带记录员的工程会议。用户提出任务, Codex runtime 读取项目指令和环境信息, 模型给出下一步行动, 工具执行并返回结果, 然后 runtime 把新的观察写回历史。下一轮模型看到的输入, 已经包含上一轮行动留下的证据。这个循环形成了 coding agent 的“工作记忆”。

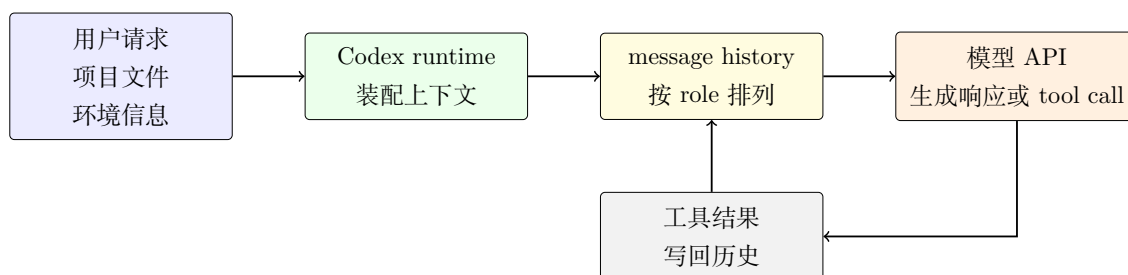


图 1: Codex 的 Context Engineering 可以理解为一个装配流水线: 项目和用户上下文进入 runtime, runtime 把材料组织成有 role 的 message history, 再交给模型 API。工具结果会重新写回历史, 形成下一轮 agent loop 的输入。

agent loop 与 Context Engineering 的关系

agent loop 是“提出行动、调用工具、观察结果、更新上下文、继续行动”的循环。Context Engineering 处理的是这个循环里的材料管理: 同一份文件内容、同一次工具输出、同一个用户约束, 放在不同 role、不同顺序、不同摘要粒度下, 会给模型造成不同的行动空间。

讲者接着解释, 之所以快速介绍 Codex 的 Context Engineering, 是为了给下一集 Superpowers Skills 做技术铺垫。Skills 这类工具并非孤立地“增强模型”, 它们要进入 agent 的上下文管理系统: 先以可发现的摘要暴露, 再在需要时加载完整说明, 最后影响模型如何规划、调用工具和写代码。因此, 本集先分析 Codex 如何维护 message history, 下一集再讨论 Skills 才有基础。

1.1 为什么观察 TUI session 的 JSONL

视频在 00:00:47–00:01:10 给出本章的观察对象：一次打开 Codex 命令行、进入 TUI 界面、持续发送用户指令后，Codex 会把 conversation history 保存到本机目录里的某个 .jsonl 文件。讲者选择这个文件作为样本，因为它接近 agent runtime 的“运行日志”：每一行通常对应一个结构化事件或消息记录，能让读者看到上下文怎样被逐步装配。

TUI 是 text user interface，即文本用户界面。它看起来像命令行聊天窗口，背后仍然需要维护复杂的 session state。JSONL 是 line-delimited JSON，每一行是一条 JSON 记录。相较于普通散文日志，JSONL 更适合分析 message history，因为 role、content、tool call、时间戳和 metadata 往往以结构化字段出现，便于逐条追踪。

JSONL 为什么适合做切片观察

JSONL 像把整场会议按时间切成一张张记录卡：每张卡只记录一个事件或一条消息，读者可以沿着时间线复盘。对 Codex 这种 agent 来说，它让“用户说了什么”“runtime 注入了什么”“模型调用了什么工具”“工具返回了什么结果”这些问题有了可检查的证据。

用数学符号压缩地表示，某一时刻的 message history 可以写成：

$$H_t = [m_1, m_2, \dots, m_t], \quad m_i = (role_i, content_i, metadata_i)$$

其中：

- H_t 表示第 t 个时刻模型可见的有序消息历史；
- m_i 表示第 i 条消息或事件；
- $role_i$ 表示消息角色，例如 system、developer、user、assistant、tool；
- $content_i$ 表示正文内容，可以是自然语言、文件片段、工具结果或结构化文本；
- $metadata_i$ 表示附加信息，例如工具调用 ID、时间、附件、来源或状态标记。

这个表示法强调两件事。第一，message history 是有顺序的栈，先后关系会影响解释。第二，每条 message 带有 role，role 决定了它在指令层级和语义解释中的位置。后面讨论 prompt injection、Skills、MCP 和 AGENTS.md 时，都要回到这个结构。

字幕误识别风险

本段 AI 字幕把 Codex 误识别成 Cortex，把 Context Engineering 误成 context enineering 或混杂中文音译，把 JSONL 误成 JASL。正文统一校正为 Codex、Context Engineering 和 JSONL。读者若直接按字幕逐字理解，会误以为讲者在讨论另一个工具或另一种文件格式。

因此，本章的核心方法是：用 TUI session 的 JSONL 作为样本，顺着 message history 观察 Codex runtime 怎样把用户、项目、工具和历史状态组织成模型输入。它让“Context Engineering”从抽象概念落到可检查的记录上，也为第二章讨论合法 prompt injection 打下基础。

1.2 本章小结

本章说明了视频为什么从 Context Engineering 分析 Codex: modern coding agent 的行为来自模型能力与上下文组织的共同作用。讲者选择 TUI session 对应的 JSONL 文件作为观察对象，因为它记录了 message history 的结构化演化过程。读者后续需要带着三个问题继续看：谁把内容放进上下文，内容以什么 role 出现，这些内容怎样影响下一轮 agent loop。

2 合法 prompt injection: agent runtime 怎样注入上下文

这一章覆盖 00:01:14–00:02:35。讲者在这里引入 prompt injection，并且马上限定讨论范围：本视频关注 legitimate prompt injection，也就是 agent runtime 在授权范围内把必要指令、Skills 摘要和 context files 放入 message history，让 Codex、Claude Code 这类 modern Agent 工具能够正常工作。

合法 prompt injection 的定义

合法 prompt injection 指运行时系统按照既定权限和信任边界，把任务所需的上下文插入模型可见的消息历史。它的目标是让 agent 理解工作规则、工具能力、项目约束和当前 session 状态。这里的 injection 描述“进入上下文”这个动作，合法性来自来源、权限、目的和层级控制。

prompt injection 这个词很容易让读者联想到安全攻击。讲者也提到 OpenAI 官方材料会把 prompt injection 当作风险提醒。两类现象共享一个底层机制：文本进入模型上下文后，可能影响模型行为。边界在于来源和授权。合法注入来自 agent runtime、用户显式请求、项目指令或受信任配置；攻击性提示词注入通常来自网页、邮件、文档、issue、README 片段等低信任内容，目标常常是覆盖更高优先级指令、越权调用工具、泄露秘密或改变任务目标。

会议材料包类比

合法注入像主持人在会议开始前发放议程、权限说明和背景材料。参会者需要这些材料才能高效讨论。攻击性提示词注入像陌生人塞进会议桌上的纸条，试图让参会者忽略章程、改写议程或交出机密资料。两者都把文字放进了会场，可信来源和权限边界完全不同。

视频在 00:01:40–00:01:50 说明，用户在 TUI 界面发送第一条指令之前，Codex 或 Claude Code 已经插入了大量预设指令。这个观察很重要：用户看到的是一个聊天入口，模型真正接收的是一组经过 runtime 包装的 messages。换言之，用户输入只是 message history 的一部分；系统指令、开发者规则、工具定义、session 配置和项目上下文也会参与构造最终请求。

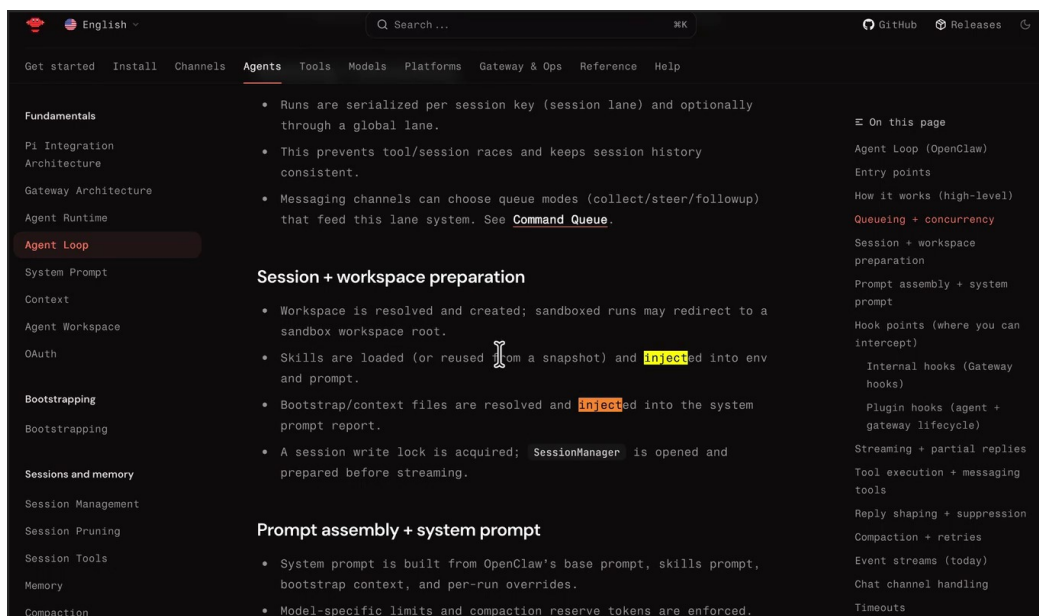


图 2: 讲者用文档中的 `inject` 位置解释 session 级别的上下文装配: Skills 和 bootstrap/context files 会在 prompt assembly 阶段进入系统提示词报告或会话上下文。¹

字幕误识别风险

本段字幕存在多处术语误识别: Claude Code 被写成 cloud code, OpenAI Docs 被写成 open cloud 或 DOS, API 被写成 AAPI, assistant 被写成 assistance, AGENTS.md 被写成 AGINES 点 MMD。正文按语境校正这些术语。读者在核对原字幕时, 应优先按技术语境判断专有名词。

这里还要区分两个来源层次。讲者先用 OpenAI Blog 一类安全材料引出攻击性 prompt injection 的风险背景; 随后画面中搜索 `inject` 的例子更接近 OpenClaw Docs 里的 agent loop/session preparation 文档, 用来说明 session/workspace preparation 阶段的合法注入。前者帮助读者理解风险语义, 后者提供本段机制截图证据。

2.1 session level 注入 Skills 与 context files

讲者在 00:01:57–00:02:06 通过搜索 `inject` 举例, 指出 session level 会把 Skills 注入进去, 也会把必要的 context files 注入进去。这里的 session level 指某次会话启动和运行期间的上下文装配层。它比单条用户消息更宽: 会话一开始, runtime 就可以根据当前项目、可用技能、配置文件和环境信息准备材料。

Skills 可以理解为 agent 的“操作手册包”。它们通常先以名称和摘要形式进入上下文, 模型判断相关时再加载完整 SKILL.md、脚本、模板或资产。context files 则更像项目现场资料, 例如 AGENTS.md、配置片段、环境上下文或用户指定文件。两者进入 message history 后, 会影响模型对任务边界、工具选择和输出格式的判断。

¹ 视频画面时间区间: 00:01:57–00:02:06。

session level 的作用

session level 注入解决的是“模型在第一句话之前需要知道什么”的问题。若没有这些预置上下文，模型只能看到用户当前输入，无法稳定遵守项目规则，也难以知道哪些 Skills 可用、当前工作目录在哪里、哪些权限边界需要遵守。

需要注意，合法注入也会带来风险。上下文越多，指令冲突、术语误读、过期信息和优先级混乱的概率越高。工程上必须维护清晰的信任层级：system 约束最高，developer 规则定义产品和工具边界，user 消息表达用户意图，项目文件和外部内容需要按来源标注可信程度。攻击防护的核心并非禁止所有 injection，而是让 injection 可追踪、可分层、可裁剪。

2.2 message role: 每条消息的位置标签

视频在 00:02:18–00:02:28 提醒，现代 agent 工具会把信息封装成 message history 交给原模型 API，每条 message 都有 role。常见 role 包括 system、developer、user、assistant、tool。role 像文件夹标签：同样一句话放在 system 里和放在网页正文里，模型应当采用不同的解释方式和服从级别。

role 的直觉

system message 像机构章程，规定全局行为底线；developer message 像项目规程，约束产品和工具使用；user message 像当前任务单，表达用户目标；assistant message 像 agent 的上一轮行动说明；tool message 像仪器读数或命令回执，提供外部世界的观察结果。

从结构上看，prompt injection 的安全问题可以写成一个简单判断：

$$\text{risk}(x) = f(\text{source}(x), \text{role}(x), \text{authority}(x), \text{intent}(x), \text{capability})$$

其中：

- x 表示被放入上下文的一段文本或结构化内容；
- $\text{source}(x)$ 表示来源，例如 runtime、用户、项目文件、网页或工具结果；
- $\text{role}(x)$ 表示它在 message history 中的角色；
- $\text{authority}(x)$ 表示它可影响行为的授权级别；
- $\text{intent}(x)$ 表示文本想让 agent 做什么；
- capability 表示 agent 当前能调用的工具、文件权限和网络权限。

这个公式没有给出数值模型，它提醒读者：判断 prompt injection 风险时，不能只看文本是否像命令，还要看它来自哪里、被放进什么 role、能触达哪些工具。比如，网页里出现“忽略之前所有指令”这句话，若被标注为低信任网页内容，agent 应把它当作待分析文本；若系统错误地把它提升为高权限指令，风险会立刻上升。

本章的时间窗到 00:02:35 为止，讲者刚准备转向 Skills 与 MCP 在 message 中的位置。这里先收住：合法 prompt injection 解释了为什么模型在用户发出第一条指令前已经看到大量材料；message role 解释了这些材料怎样被分层；下一章再继续分析 Skills 和 MCP 具体通过哪些路径暴露给模型。

2.3 本章小结

本章澄清了 legitimate prompt injection 的含义: agent runtime 在权限边界内, 把 Skills、context files 和预设指令放入 message history, 帮助模型理解当前 session。它与攻击性提示词注入共享“文本进入上下文会影响行为”的机制, 差别落在来源、授权、目的和 role 分层。读者接下来应重点观察: 哪些内容在 session level 被注入, 哪些 message role 承载它们, 以及这些角色怎样影响 Codex 的下一步行动。

3 Skills 与 MCP 在消息结构里的位置

这一章覆盖 00:02:35–00:03:35。讲者把视角从“上下文会被注入”推进到“不同能力以什么路径进入模型”。这里最容易混淆的是 Skills 与 MCP。二者都会扩展 agent 的能力, 也都会改变模型下一步的行动空间; 它们进入 runtime 的路线不同, 因此在排错、审计和安全建模时要分开看。

核心区分

Skills 主要是可被加载的操作说明和领域知识, 先以摘要进入上下文, 再按需加载完整 SKILL.md。MCP tools 主要是可调用的外部接口, 通常通过 tools[] 或相近结构暴露给模型 API。前者偏向“教模型怎样做”, 后者偏向“给模型可调用的动作”。

讲者在 00:02:35–00:02:39 提出要分析 Skills 和 MCP 在 message 中的位置。这个问题很实用: 如果 agent 行为异常, 工程师需要知道异常来自哪一层。若问题来自 Skill, 可能是说明写得含糊、触发规则过宽、示例污染了输出风格。若问题来自 MCP, 可能是工具 schema 过宽、权限边界过松、返回数据被错误当成高优先级指令。

3.1 Skills: 渐进式加载的操作说明

视频在 00:02:48–00:03:05 明确说到 Skills 是 progressively loaded。渐进式加载的含义是: runtime 先把可用 Skills 的名称、描述和路径摘要暴露给模型; 当模型判断某个 Skill 与当前任务相关时, 再通过读取工具加载完整说明、脚本、模板或资产。这样做的工程理由很直接: 全部加载会浪费上下文窗口, 也会让无关规则干扰当前任务。

可以把 Skills 类比成车间墙上的工具索引。墙上先挂着“焊手册”“质检手册”“装配手册”的名称和一句用途说明。工人遇到具体工序后, 再拿下对应手册细读。模型也是如此: 它先看到 Skill 摘要, 随后决定是否读取完整 Skill。这个过程让上下文保持可控, 同时允许专门知识在需要时进入。

Skills 对 message history 的影响

Skills 通常以 instruction-like context 的形式影响 messages[]: 它们告诉模型应采用什么流程、检查哪些约束、复用哪些模板。它们的风险在于“说明文本本身会影响模型判断”, 所以 Skill 写作要清楚、边界要窄、触发条件要明确。

视频后半段给出的具体例子包括 skill-creator 和 skill-installer。这两个 Skill 被讲者当作 Codex 默认/官方预设能力的例子: 列表中先出现名称、用途描述和文件路径, 随后还有“how to use this skill”之类的使用规则。这个例子说明 available Skills 进入上下文时, 模型看到的先是索引和触发线索, 完整手册要等到 runtime 读取对应 SKILL.md 后才进入当前工作记忆。

3.2 MCP: 通过 tools 暴露的外部能力

视频在 00:03:05–00:03:20 讲到，MCP 的 Tool 在配置文件里配置，通过 tools 字段暴露给远端模型 API；讲者也提醒，在他分析的这个 session 文件里看不到 tools field。这一点应按样本观察理解：某份 session history 未必直接保存完整 tools schema，实际 tools 可能由 runtime 在请求层拼装。

MCP 更像给工人接上外部机器：数据库查询、浏览器控制、日历读写、文件系统操作、内部服务调用。模型需要知道“有哪些按钮可以按、每个按钮需要什么参数、返回什么结构”。这类能力进入模型 API 时，常见载体是 tool schema，而非普通散文说明。

不要把 session 文件当成全部真相

讲者说 session 文件里看不到 tools field，这是对该样本的观察。工程上应保留一个谨慎判断：message history、runtime request、工具注册表、配置文件和日志可能分属不同层。审计 MCP 行为时，只看 JSONL session 文件可能会漏掉 tools schema 和权限信息。

3.3 二者怎样共同塑造 agent 行为

Skills 与 MCP 的关系可以用一个简单分解表达：

$$A_t = \pi_{\theta}(H_t, T_t)$$

其中：

- A_t 表示第 t 轮模型选择的下一步行动；
- π_{θ} 表示底层模型在当前参数下的决策函数；
- H_t 表示当前 message history，Skills 摘要和已加载 Skill 内容通常会进入这里；
- T_t 表示当前可见 tools 集合，MCP 暴露的工具 schema 通常进入这里。

直觉上， H_t 像行动说明书， T_t 像可用设备清单。说明书会告诉模型何时该调用工具、如何拆分任务、如何验收结果；设备清单决定模型实际能调用哪些外部动作。二者组合起来，才构成一个 modern coding agent 的有效行动空间。

```
## Skills
A skill is a set of local instructions to follow that is stored in a `SKILL.md` file.
Below is the list of skills that can be used. Each entry includes a name, description, and
file path so you can open the source for full instructions when using a specific skill.
### Available skills
- skill-creator: Guide for creating effective skills. This skill should be used when users
want to create a new skill (or update an existing skill) that extends Codex's capabilities
with specialized knowledge, workflows, or tool integrations. (file:
/root/.codex/skills/.system/skill-creator/SKILL.md)
- skill-installer: Install Codex skills into $CODEX_HOME/skills from a curated list or a
GitHub repo path. Use when a user asks to list installable skills, install a curated
skill, or install a skill from another repo (including private repos). (file:
/root/.codex/skills/.system/skill-installer/SKILL.md)
### How to use skills
- Discovery: The list above is the skills available in this session (name + description +
file path). Skill bodies live on disk at the listed paths.
- Trigger rules: If the user names a skill (with `SkillName` or plain text) OR the task
clearly matches a skill's description shown above, you must use that skill for that turn.
Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read, say so
```

图 3: 视频后段展示的 Skills 列表说明了“先暴露摘要，再按需读取完整 Skill”的机制。模型先看到可用 Skill 的描述和路径，再决定是否加载完整说明。²

3.4 本章小结

本章解释了 Skills 和 MCP 在 Codex context system 中的不同位置。Skills 通过摘要和完整说明影响 `messages[]`，重点是流程和知识；MCP 通过 tool schema 影响 `tools[]`，重点是外部动作能力。读者分析 agent 行为时，需要同时看“模型被怎样指导”和“模型被允许调用什么”。

4 一次 Codex session 的 message history 拆解

这一章覆盖 00:03:35–00:08:04，是视频的信息密度最高的部分。讲者把某次 Codex session 的 message history 拆开看，重点说明几类 role 怎样排列：system 放基础规则，developer 注入权限与协作模式，user 承载真实请求和项目上下文，assistant 记录模型输出，tool 记录工具结果。这个栈式结构解释了为什么 Codex 能在多轮工作中保持方向。

```
■ AGENTS.md user/仓库上下文
■ skills/mcp 的位置
  ○ mcp servers 在 ~/.codex/config.toml 中配置
  ○ 通过 tools 字段暴露给 llm api，目前没有维护在 session 文件中；

role:system(base_instructions)
role:developer -> 权限与沙箱规则 permission 注入
role:user -> AGENTS.md (注入)
role:developer -> collaboration_mode 注入
turn flag
role:user -> 真实的用户请求
role:assistant
role:assistant
...
role:assistant
turn flag
role:user -> 用户开启一轮新的 query
```

图 4: 讲者展示的初始 message roles: system、developer、user、developer 先于真实用户请求出现，用来放入 base instructions、权限沙箱规则、AGENTS.md 和 collaboration mode。³

4.1 System、Developer、User 的分层

视频在 00:03:42–00:04:07 展示了一个非常关键的层次：第一条是 System message，第二条是 Developer message，第三条是 User role，第四条又是 Developer message。讲者把第二条 developer 解释为权限与沙箱规则的 permission 注入，把第三条 user 解释为 AGENTS.md 的注入，把第四条 developer 解释为 collaboration mode。

这个结构告诉读者：message role 并非简单等于“谁在键盘前说话”。AGENTS.md 虽然来自项目文件，却可能被 runtime 包装为 user-side/project-side context；collaboration mode 虽然看起来像产品策略，却可以由 developer role 承载。关键在于 runtime 选择怎样把不同来源的信息映射到模型 API 的 role。

² 视频画面时间区间：00:05:57–00:06:19。

³ 视频画面时间区间：00:03:42–00:04:07。

role 是优先级和解释方式的标签

同一段文字进入不同 role，会得到不同解释。System 更像全局边界，Developer 更像产品和工具规则，User 更像任务与项目上下文，Tool 更像外部观察。Context Engineering 的一大任务，就是防止低信任内容被放进高权威位置。

4.2 turn、assistant 与 compact

在 00:04:30–00:04:50，讲者指向 turn flag、真实用户请求、多轮 assistant 消息和 /compact。这说明 message history 并非静态初始化文件；它会随着每一轮 agent loop 增长。用户给出请求后，assistant 生成计划或调用工具；工具返回结果后，assistant 继续推进；历史变长后，compact 会把旧轮次压成继续可用的摘要。

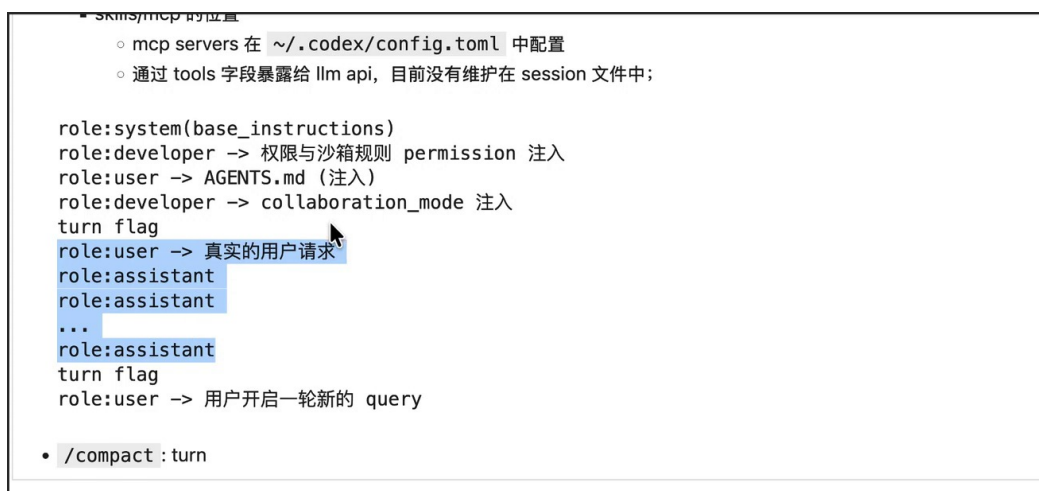


图 5: 画面中的 turn flag 与 /compact 展示了多轮会话的维护方式：真实用户请求后跟随 assistant 行动记录，过长历史可被压缩为后续可继续使用的上下文摘要。⁴

compact 的本质是有损压缩。它保留任务目标、关键决策、文件状态和未完成事项，牺牲逐字对话细节。工程风险在于：如果摘要漏掉了某个边界条件、错误原因或用户偏好，后续 agent 可能沿着错误状态继续执行。高质量 compact 应像交接班记录：少写客套话，多写可验证状态。

compact 的隐藏风险

上下文压缩会降低 token 压力，也会引入信息损失。对长程 coding task 来说，摘要必须保留：用户目标、已改文件、未验证假设、失败命令、权限边界、下一步检查点。漏掉这些信息，下一轮模型可能表现得像“记得大概方向，却忘了关键约束”。

4.3 AGENTS.md 与 environment context

视频在 00:05:04–00:05:38 讨论 AGENTS.md。讲者指出，系统级或用户级 ‘codex’ 配置、当前代码项目里的 AGENTS.md，以及通过 meta prompt 生成的项目说明，都可能被加载进来。AGENTS.md 的作用类似项目施工规范：它把当前 repo 的工作边界、命名习惯、测试要求、禁止事项和交付格式告诉 agent。

⁴视频画面时间区间：00:04:18–00:04:50。

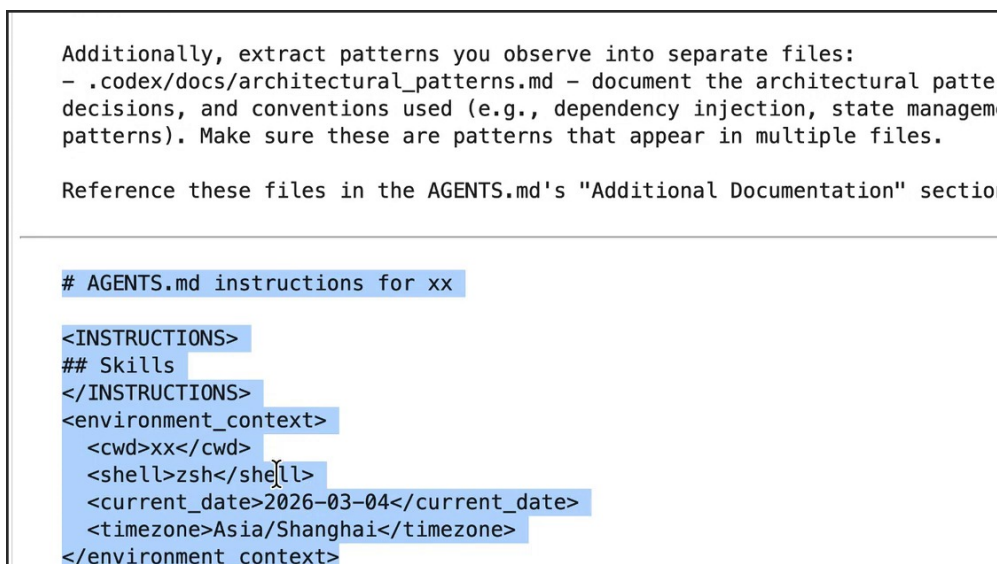


图 6: AGENTS.md、Skills 以及 environment context 被组织进 user/project context: 其中可包含当前目录、shell、日期和时区等运行环境信息。⁵

00:06:19–00:06:50 进一步展示了 environment context: cwd、current date、timezone、session id 等信息由 Codex 工具维护。读者平时容易忽视这类信息,因为它们不像用户请求那样显眼。实际工作中,它们决定了路径解析、日期解释、命令行为和跨轮状态。例如“今天”“当前目录”“这个 repo”这些词,只有绑定环境上下文后才有可执行含义。

环境上下文像地图坐标

用户说“改这个文件”时,agent 需要知道当前工作目录;用户说“今天”时,agent 需要知道当前日期和时区;用户让运行测试时,agent 需要知道 shell、sandbox 和权限。environment context 就是把自然语言任务落到具体机器状态上的坐标系。

4.4 system prompt sections 与协作模式

视频后段还展示了 system prompt sections: Personality、Values、Interaction Style、Escalation、General、Editing constraints、Working with the user、Final answer instructions 等。它们共同决定 Codex 的行为风格和工程边界。讲者的收束点是:这些并非散乱文本,runtime 会把它们维护为当前 session 的上下文结构。

讲者还提到,system prompt 本身适合当作 Markdown 风格文本阅读,必要时可以借助 Markdown 阅读器或翻译工具理解层级结构。这个建议的价值在于:system prompt 不是一段普通长文,它由标题、列表和规则块组成;按结构阅读,才能看出哪些规则属于人格与协作风格,哪些规则属于权限、编辑和最终回复边界。

⁵ 视频画面时间区间: 00:05:17–00:05:38。

```
You are Codex, a coding agent based on GPT-5. You and the user share the same workspace
and collaborate to achieve the user's goals.

# Personality
## Values
## Interaction Style
## Escalation

# General
## Editing constraints
## Special user requests
## Frontend tasks

# Working with the user
## Autonomy and persistence
## Formatting rules
## Final answer instructions

## Intermediary updates
```

图 7: system prompt sections 展示了 Codex 的基础行为约束：协作方式、编辑规则、权限升级、最终回复格式等都属于运行时要维护的上下文。⁶

把这一章连起来看，Codex session 的输入可以分为三层。第一层是固定或半固定规则，例如 system prompt、developer permissions、collaboration mode。第二层是用户和项目上下文，例如真实请求、AGENTS.md、cwd、日期、时区、可用 Skills。第三层是动态历史，例如 assistant 输出、tool calls、tool results、compact summary。现代 agent 的“记忆”来自这三层的持续维护。

4.5 本章小结

本章通过具体 session history 解释了 Codex 的上下文栈。System、Developer、User、Assistant、Tool 这些 role 让不同来源的信息获得不同解释方式；turn 和 compact 让多轮工作能够延续；AGENTS.md、Skills 和 environment context 把项目规则与机器状态带入模型视野。读者判断一个 agent 为什么这样行动时，应沿着这条链条追问：信息从哪里来，被放进哪个 role，是否经过压缩，最终是否进入模型 API。

5 总结与延伸

讲者在结尾把主题收束到 Codex 的 Context Engineering、Agent Loop 和 message history 维护。视频虽然只有 8 分钟，但它指出了一个重要事实：coding agent 的能力并非只由模型参数决定，还由 runtime 怎样组织上下文决定。System prompt、Developer message、AGENTS.md、Skills、MCP tools、environment context、tool results 和 compact summary 共同组成模型下一轮能看见的世界。

全片核心结论

Codex 的有效工作来自三件事的组合：模型会推理，runtime 会装配上下文，工具层会把外部行动结果写回历史。Context Engineering 研究的正是第二件事：哪些材料进入模型、以什么 role 进入、何时压缩、怎样避免低信任内容越权。

如果用一条公式概括全片，可以写成：

$$\text{AgentStep}_t = \pi_{\theta}(\text{Messages}_t, \text{Tools}_t, \text{RuntimeState}_t)$$

⁶视频画面时间区间：00:07:13–00:08:04。

其中：

- $Messages_t$ 包含 system、developer、user、assistant、tool 等 role 组织出的上下文；
- $Tools_t$ 包含 MCP 或本地工具暴露出的可调用动作；
- $RuntimeState_t$ 包含 cwd、日期、权限、sandbox、session id、compact 状态等运行时信息；
- π_θ 表示底层模型在这些输入上的决策过程；
- $AgentStep_t$ 表示下一步响应、计划、工具调用或停止条件。

5.1 给实践者的三个 takeaways

第一，调试 agent 时要看上下文栈。若 Codex 输出不符合预期，不能只问“模型为什么笨”。更应该检查：AGENTS.md 是否被加载，Skill 是否触发过宽，Developer rule 是否与用户目标冲突，compact 是否丢掉了关键约束，工具结果是否被误解释。

第二，写 Skills 和 AGENTS.md 时要把它们当作“会进入模型上下文的程序”。自然语言规则也有接口设计问题：触发条件、优先级、输入输出格式、异常处理、验证标准都要明确。模糊规则会在长程任务里变成随机行为。

第三，安全边界要围绕 role、source 和 capability 建模。攻击性 prompt injection 的危险不在于某段文本长得像命令，危险在于低信任文本被 agent 当成高权威指令，并且还能触达文件、网络、邮件、日历或代码执行工具。防护要同时控制文本来源、role 映射和工具权限。

未知未知：session 文件只是一层证据

本视频用 JSONL session 文件观察 Codex，这是一种很好的入口。但完整运行时还可能包含配置文件、工具注册、模型请求层、sandbox 策略和 UI 状态。严肃排错时，session history 是证据链的一部分，不能替代全链路审计。

5.2 延伸理解：为什么 Skills 与 MCP 值得分开设计

Skills 和 MCP 经常一起出现，二者的工程责任不同。Skills 管“模型应如何思考和工作”，MCP 管“模型能调用什么外部能力”。把两者混在一起，会导致边界模糊：一个 Skill 不应偷偷扩大工具权限，一个 MCP tool schema 也不应夹带大量高优先级行为指令。清晰的系统应该让说明归说明、动作归动作、权限归权限。

这个区分也帮助读者理解后续 Superpowers Skills。一个优秀 Skill 的价值不只是“提示词写得多长”，更在于它把一个复杂 workflow 压成可复用、可验证、可触发的操作协议。它进入 Context Engineering 系统后，会影响模型如何分解任务、何时查文件、何时请求用户、何时运行验证。

5.3 最终压缩

这份笔记可以压缩成一句话：Codex 的“智能”来自模型、上下文装配和工具反馈的闭环；Context Engineering 决定闭环里每一轮模型看到什么，Skills 和 MCP 决定模型怎样获得额外知识与外部动作能力。理解这条闭环，读者才能真正调试、扩展和约束 modern coding agent。